

Массивы (Arrays)

Массив (array) — упорядоченная структура данных для хранения набора элементов. Элементы могут быть любого типа (числа, строки, булевы значения и т. д.), и не обязательно одного типа.

Создаётся через литерал массива — квадратные скобки []:

```
const planets = ['Earth', 'Mars', 'Venus']; // массив строк
const numbers = [1, 2, 3, 4, 5]; // массив чисел
const mixed = ['apple', 10, true]; // массив разных типов
```

Доступ к элементам (Element Access)

Каждый элемент имеет индекс (index) — порядковый номер, начиная с 0. Доступ — через квадратные скобки без пробела перед ними: `arrayName[index]`.

```
const planets = ['Earth', 'Mars', 'Venus'];
console.log(planets[0]); // 'Earth'
console.log(planets[1]); // 'Mars'
```

Обращение к несуществующему индексу возвращает `undefined`.

Переопределение элемента (Reassigning an Element)

Элемент можно изменить, обратившись по индексу и присвоив новое значение. Сама переменная-массив при этом не меняется — меняется содержимое внутри неё.

```
const planets = ['Earth', 'Mars', 'Venus', 'Uranus'];
planets[0] = 'Jupiter';
console.log(planets); // ['Jupiter', 'Mars', 'Venus', 'Uranus']
```

Длина массива (length)

Свойство `length` хранит количество элементов и обновляется автоматически при добавлении/удалении:

```
const planets = ['Earth', 'Mars', 'Venus'];
console.log(planets.length); // 3
```

Индекс последнего элемента (Last Element Index)

Формула: `length - 1` (т. к. индексация начинается с нуля).

```
const lastElementIndex = planets.length - 1;
console.log(planets[lastElementIndex]); // 'Venus'
```

Удобно, когда длина массива заранее неизвестна.

Присвоение по значению и по ссылке (By Value vs By Reference)

Ключевое отличие сложных типов от примитивов.

Примитивы (`string`, `number`, `boolean`, `null`, `undefined`) копируются по значению (`by value`) — при присвоении создаётся независимая копия.

```
let a = 5;
let b = a;
a = 10;
console.log(a); // 10
console.log(b); // 5 — не изменилась, это отдельная копия
```

Массивы (сложный тип) присваиваются по ссылке (by reference) — переменная хранит не сами данные, а указатель на место в памяти. Если несколько переменных ссылаются на один массив, изменение через любую из них видно во всех остальных:

```
const a = ['Mango', 'Poly'];
const b = a; // b получает ссылку на тот же массив

a[1] = 'Jacob';
console.log(a); // ['Mango', 'Jacob']
console.log(b); // ['Mango', 'Jacob'] — изменился тоже
```

Важное следствие: два массива никогда не равны через ===, даже если содержимое одинаковое — сравниваются ссылки, а не содержимое.

```
console.log([1, 2, 3] === [1, 2, 3]); // false
console.log([] === []); // false
```

Приведение типов: массивы (Type Coercion: Arrays)

Массив → строка (String)

Элементы объединяются через запятую.

```
console.log(String([1, true, 'Poly'])); // "1,true,Poly"
```

Массив → число (Number)

Сначала массив приводится к строке, затем строка — к числу. Пустой массив → 0, массив с одним числовым элементом → это число, массив с несколькими элементами → NaN.

```
console.log(Number([])); // 0
console.log(Number([1])); // 1
console.log(Number([1, 2, 3])); // NaN
```

Массив → булево значение (Boolean)

Любой массив, включая пустой, приводится к true.

```
console.log(Boolean([])); // true
```

Методы массива (Array Methods)

`join(delimiter)`

Объединяет элементы массива в строку, вставляя между ними указанный разделитель (delimiter).

```
const words = ["JavaScript", "is", "amazing"];
console.log(words.join("")); // 'JavaScriptisamazing'
console.log(words.join(" ")); // 'JavaScript is amazing'
console.log(words.join("-")); // 'JavaScript-is-amazing'
```

Полезно, например, для конвертации snake_case (змеиная нотация с подчёркиваниями) в kebab-case (нотация с дефисами):

```
function transformString(string) {
  const words = string.split("_");
  return words.join("-");
}
transformString("user_age"); // "user-age"
```

split(delimiter)

Метод строк (не массива!) — обратная операция к join(). Разбивает строку на массив по указанному разделителю. Пустой разделитель "" даёт массив отдельных символов.

```
console.log("Mango".split("")); // ["M", "a", "n", "g", "o"]
console.log("JavaScript essentials".split(" ")); // ["JavaScript", "essentials"]
console.log("amazing-french-recipes".split("-")); // ["amazing", "french", "recipes"]
```

slice(begin, end)

Возвращает новую копию части массива — от индекса begin до end не включительно. Исходный массив не меняется.

```
const planets = ["Earth", "Mars", "Venus", "Jupiter", "Saturn"];
console.log(planets.slice(0, 2)); // ['Earth', 'Mars']
console.log(planets.slice(1, 3)); // ['Mars', 'Venus']
```

Особые случаи:

- без параметров — полная копия массива: planets.slice()
- без end — копирует от begin до конца: planets.slice(1) → ["Mars", "Venus", "Jupiter", "Saturn"]
- отрицательный begin без end — копирует последние N элементов: planets.slice(-2) → ["Jupiter", "Saturn"]

concat(arr1, arr2, ...)

Объединяет два или более массивов в новый, не изменяя исходные. Порядок аргументов определяет порядок элементов в результате.

```
const firstArray = ["Mercury", "Venus"];
const secondArray = ["Mars", "Jupiter"];
const result = firstArray.concat(secondArray);
console.log(result); // ["Mercury", "Venus", "Mars", "Jupiter"]
console.log(firstArray); // ["Mercury", "Venus"] — не изменился
```

indexOf(elem)

Возвращает индекс первого вхождения элемента в массиве, либо -1, если не найден. Сравнение строгое (===).

```
const clients = ["Mango", "Ajax", "Poly", "Kiwi", "Poly"];
console.log(clients.indexOf("Poly")); // 2 (первое вхождение)
console.log(clients.indexOf("Monkong")); // -1
```

```
push(elem1, elem2, ...)
```

Добавляет один или несколько элементов в конец массива. Изменяет исходный массив.

```
const planets = ["Earth", "Mars", "Venus"];
planets.push("Jupiter");
console.log(planets); // ['Earth', 'Mars', 'Venus', 'Jupiter']

planets.push("Saturn", "Neptune");
console.log(planets); // [..., "Jupiter", "Saturn", "Neptune"]
```

Часто используется в цикле для динамического наполнения массива:

```
const tags = [];
for (let i = 0; i < 3; i += 1) {
  tags.push(`tag-${i}`);
}
console.log(tags); // ["tag-0", "tag-1", "tag-2"]
```

Итерация по массиву (Array Iteration)

Цикл for

Позволяет перебрать элементы массива и выполнить действие для каждого.

```
const planets = ["Earth", "Mars", "Venus"];

for (let i = 0; i < planets.length; i += 1) {
  console.log(planets[i]);
}
```

- счётчик `i` стартует с 0, цикл идёт пока `i < planets.length`
- на каждой итерации `planets[i]` даёт доступ к элементу по индексу
- `i` увеличивается на 1 каждый раз (`i += 1`)

includes(element)

Проверяет, есть ли элемент в массиве. Возвращает true/false.

```
const planets = ["Earth", "Mars", "Venus"];
console.log(planets.includes("Earth")); // true
console.log(planets.includes("Jupiter")); // false
```

Часто используется в условиях:

```
const fruits = ["apple", "banana", "orange"];

if (fruits.includes("banana")) {
  console.log("The array has an element banana");
} else {
  console.log("Array does not contain banana element");
}
```

Цикл for...of

Более удобный способ перебора массива, когда индекс (счётчик) не нужен.

```
for (const element of array) {  
  // тело цикла  
}
```

Пример:

```
const planets = ["Earth", "Mars", "Venus"];  
  
for (const planet of planets) {  
  console.log(planet);  
}
```

`for...of` всегда проходит массив от первого до последнего элемента — условие остановки задать нельзя. Чтобы прервать цикл досрочно, используется `break`.

Функции (продолжение)

Псевдомассив `arguments`

Внутри функции автоматически доступна специальная переменная `arguments` — она содержит все переданные при вызове аргументы.

```
function sum(a, b) {  
  console.log(arguments);  
  return a + b;  
}  
  
sum(2, 5);
```

Это псевдомассив (pseudo-array):

- есть свойство `length` и доступ по индексу, как у массива
- нет методов массива (`push`, `map` и т. д.)
- можно перебирать циклами

```
function multiply() {  
  let total = 1;  
  for (const arg of arguments) {  
    total *= arg;  
  }  
  return total;  
}  
  
console.log(multiply(1, 2, 3)); // 6  
console.log(multiply(1, 2, 3, 4)); // 24
```

Если нужны именно методы массива — превращаем `arguments` в настоящий массив через `Array.from()`:

```
function foo() {  
  const args = Array.from(arguments);  
  return args.join("-");  
}  
  
foo(1, 2, 3); // "1-2-3"
```

Параметры по умолчанию (Default Parameters)

Параметр может иметь значение по умолчанию — оно применяется, если аргумент не передан при вызове.

```
function greet(username = "Guest") {  
  console.log(`Hello, ${username}!`);  
}  
greet("Jacob"); // "Hello, Jacob!"  
greet(); // "Hello, Guest!"
```

```
function count(from, to, step = 1) {  
  console.log(`from: ${from}, to: ${to}, step: ${step}`);  
}  
count(1, 15, 4); // "from: 1, to: 15, step: 4"  
count(1, 15); // "from: 1, to: 15, step: 1"
```

Функциональное выражение (Function Expression)

Альтернативный способ объявления функции — обычная переменная, значением которой является функция.

Function declaration (объявление функции):

```
function multiply(x, y, z) {  
  console.log(x * y * z);  
}
```

Function expression (функциональное выражение):

```
const multiply = function (x, y, z) {  
  console.log(x * y * z);  
};
```

Ключевое отличие: function expression нельзя вызвать до места объявления (это обычная const), а function declaration — можно, она поднимается (hoisting) целиком.

```
// ❌ Ошибка — вызов до объявления  
multiply(1, 2, 3);  
const multiply = function (x, y, z) { console.log(x * y * z); };
```

```
// ✅ Работает — function declaration можно вызвать раньше объявления  
multiply(1, 2, 3);  
function multiply(x, y, z) { console.log(x * y * z); }
```

Важно не смешивать оба стиля в одном проекте — код должен быть единообразным.

Область видимости (Scope)

Scope — механизм, определяющий, где в коде видна переменная.

Цепочка областей видимости (scope chain): вложенные (дочерние) области видят переменные родительских областей, но не наоборот.

Глобальная область видимости (global scope) — переменные, объявленные вне любых блоков {}, доступны везде:

```
const globalValue = 10;  
  
function foo() {  
  console.log(globalValue); // 10  
}
```

Блочная область видимости (block scope) — переменные, объявленные внутри {} (в if, for, функции), доступны только внутри этого блока и вложенных в него блоков:

```
function foo() {  
  const a = 20;  
  console.log(a); // 20  
}  
console.log(a); // ❌ Ошибка — a недоступна здесь
```

Поиск переменной идёт изнутри наружу: сначала в текущей области, затем в родительских, вплоть до глобальной. Если нигде не найдена — ошибка "переменная не объявлена".

Есть зарезервированные глобальные имена, например name. Если объявить одноимённую переменную в блоке — внутри блока будет использоваться локальная, а снаружи по-прежнему глобальная, без ошибки.

Стек вызовов (Call Stack)

JavaScript — однопоточный (single-threaded) язык: в один момент времени выполняется только одна инструкция. Когда функция вызывает другую функцию, первая ждёт завершения второй.

Стек (stack) — структура данных LIFO (Last-In-First-Out — последним пришёл, первым вышел): добавлять и удалять можно только с вершины.

Стек вызовов (call stack) отслеживает, какие функции сейчас выполняются:

- при вызове функция добавляется в стек
- функции, вызванные изнутри неё, добавляются поверх и выполняются
- по завершении функция снимается со стека, выполнение возвращается туда, где остановилось

Stack frame (кадр стека) — запись в стеке при вызове функции, хранит служебную информацию (имя функции, строку вызова).

```
function bar() { console.log("bar"); }  
function baz() { console.log("baz"); }  
function foo() {  
  console.log("foo");  
  bar();  
  baz();  
}  
foo();
```

Стек ограничен по размеру. Если функции вызывают друг друга бесконечно (например, рекурсия (recursion) без выхода), возникает переполнение стека (stack overflow) с ошибкой "Uncaught RangeError: Maximum call stack size exceeded".